

4 : Arithmetic for Computers

Rajesh Panicker, NUS

CG3207

Acknowledgement :

- Text by Patterson and Hennessey and companion slides by Mary Jane Irwin
- Text and companion slides by Harris and Harris
- Some slides from A/Prof. Bharadwaj Veeravalli
- Some figures from Wikipedia

Note:

- Not all slides will be covered in the lecture. The rest are left as a self-learning exercise

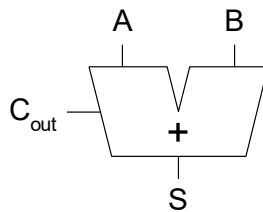


Contents

- Adder, Carry lookahead adder, Subtractor
- ALU with ADD/SUB/AND/OR
- Shifter
- Multiplier, Divider
- Numbers with fractions: Fixed-point and Floating-point arithmetic

1-Bit Adders

Half Adder

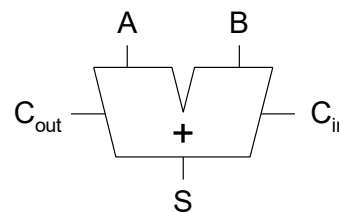


A	B	C_{out}	S
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

$$S = A \oplus B$$

$$C_{out} = AB$$

Full Adder



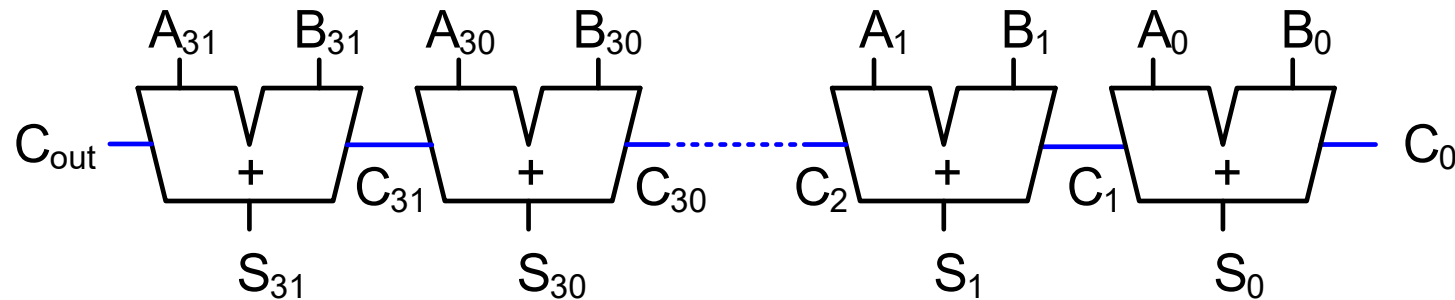
C_{in}	A	B	C_{out}	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

$$S = A \oplus B \oplus C_{in}$$

$$C_{out} = AB + AC_{in} + BC_{in}$$

Ripple-Carry Adder

- Chain 1-bit adders together
- Carry ripples through entire chain
- Advantage : simple hardware
- Disadvantage: slow (could be a part of the critical path)





Carry-Lookahead Adder

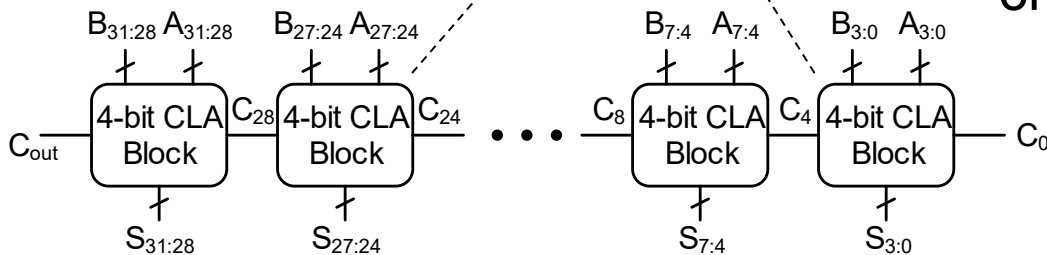
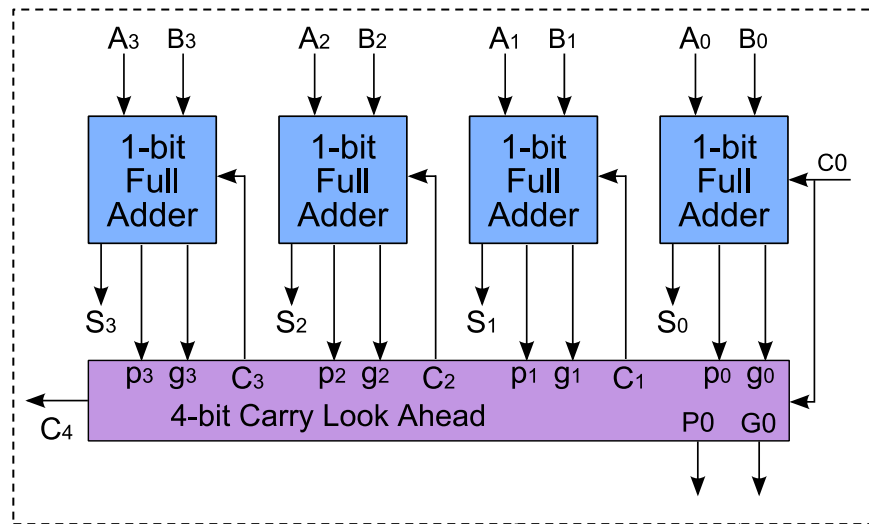
- Possible solution : express and implement C_{i+1} directly in terms of $(A_0, \dots, A_i, B_0, \dots, B_i, C_0)$ instead of waiting for it to ripple in
 - The number of terms would be huge for higher C_{i+1} s, implying huge number of gates, with each gate having a high fan-in
 - Fan-in / fan-out is the number of inputs / outputs driving / driven by a gate. Large fan-in/out usually means the gate will have a larger delay
- Carry-lookahead - understanding when we would generate a carry and when we would propagate

Carry-Lookahead Adder

- Idea : compute C_{i+1} from a pre-processed form of inputs. Observe the following
 - $C_{i+1}=1$ when $A_i = B_i = 1$ regardless of C_i . Stage i *generates* a carry. $g_i = A_i \cdot B_i$
 - $C_{i+1}=1$ when $C_i = 1$ and either $A_i = 1$ or $B_i = 1$. Stage i *propagates* a carry. $p_i = A_i + B_i$
 - $C_1 = p_0 C_0 + g_0$
 - $C_2 = p_1 C_1 + g_1 = p_1 p_0 C_0 + p_1 g_0 + g_1$
- We can continue this ... but
- How far can we go?
- Doesn't this defeat our purpose?

Carry-Lookahead Adder

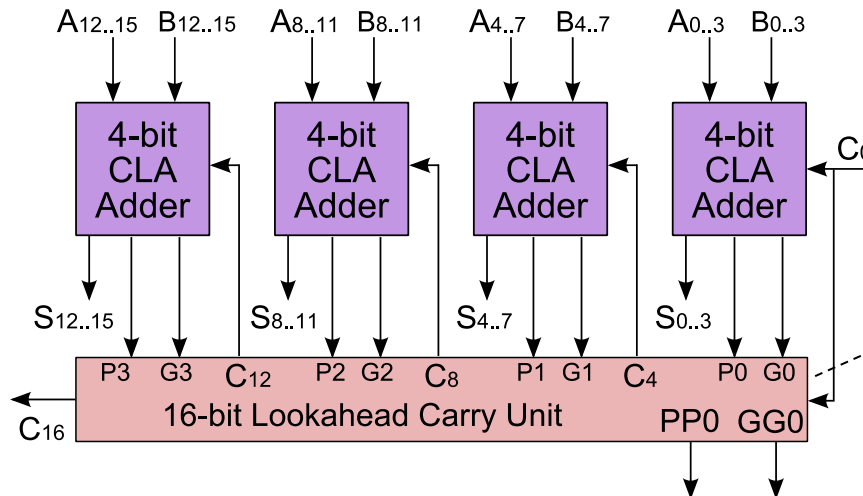
- Doing this for 32 stages is cumbersome. Do this for, say, 4 stages and call this 4 stage unit as a fundamental block
 - The first block takes $A_0, B_0, \dots, A_3, B_3, C_0$ and gives $(S_0, S_1, S_2, S_3, C_4)$



- Use 8 such units to handle 32 bits
- The ripple effect is avoided within a block, but the ripple carry effect will be present between blocks
- Nevertheless, the delays are significantly lower than the original ripple carry adder
- A better solution?

Carry-Lookahead Adder

- Block propagate (P_i) and generate (G_i) signals for 4-bit blocks
 - $P_0 = p_3 p_2 p_1 p_0$ (i refers to the block number here)
 - $G_0 = g_3 + p_3 (g_2 + p_2 (g_1 + p_1 g_0))$
- 16-bit adder using 4-bit CLAs



Identical logic as the 4-bit carry lookahead unit !

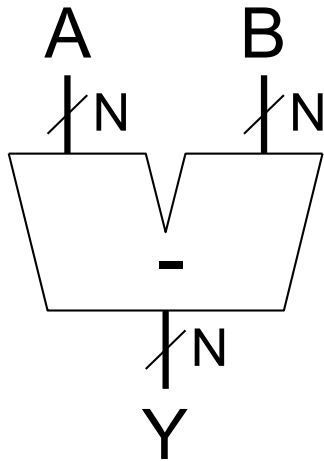
- How about 32 bit?
 - Cascade 2 x 16-bit units
 - Or ... ?

FPGAs usually have some support for carry lookahead and/or dedicated cascading routes (i.e., carry need not use the general purpose routing fabric, allowing for fast propagation). Read up about CARRY4 in AMD FPGAs. This will be used when you have a '+' in your code. Implementing your own carry look ahead logic will make your circuit slower!

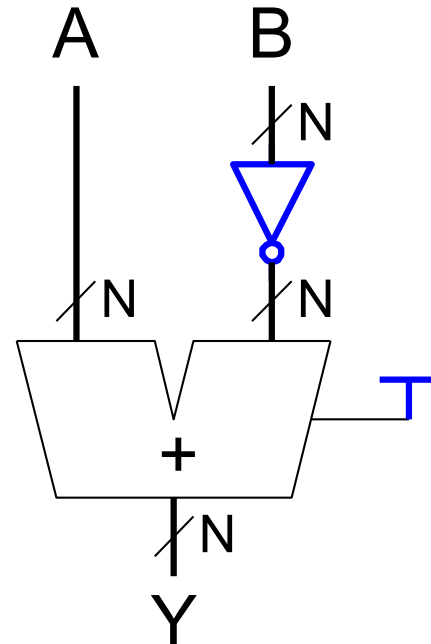
Subtractor

- Basic idea : $A - B = A + B' + 1$

Symbol

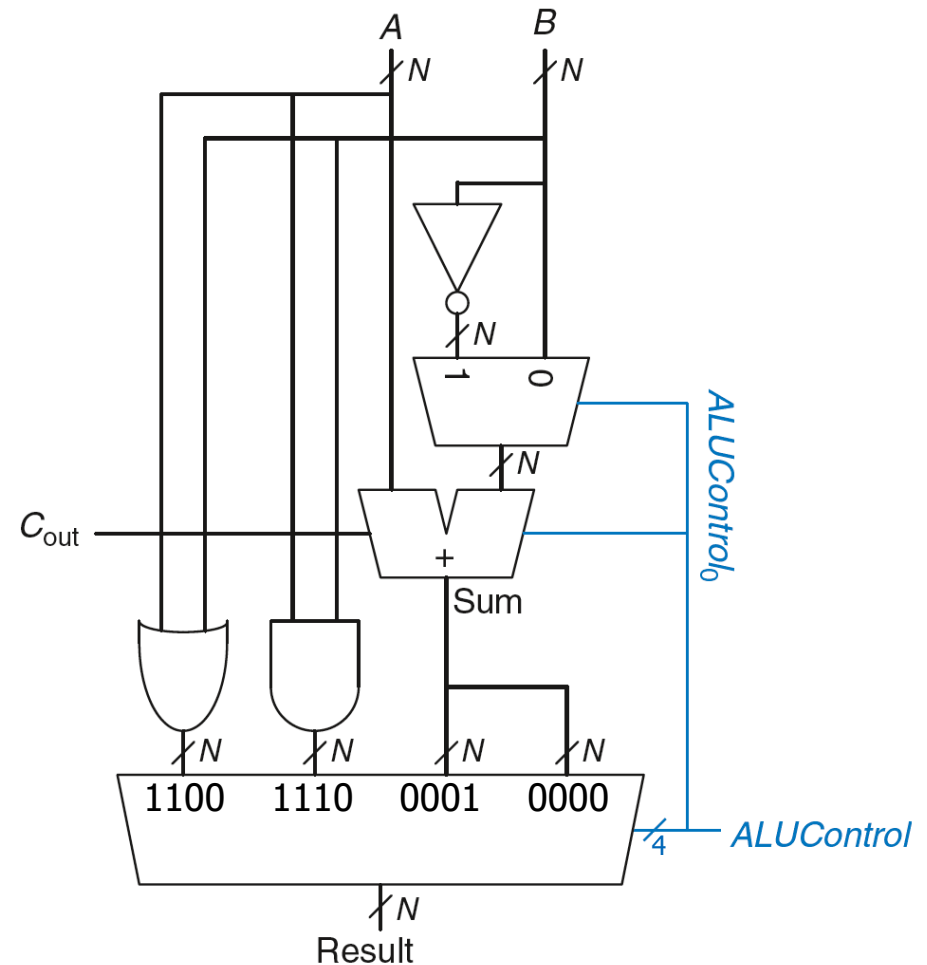


Implementation



ALU

ALUControl _{3:0}	Function
0000	ADD
0001	SUB
1110	AND
1100	OR



The first mux is simply an XOR gate. Its output = $B \wedge \text{ALUControl}_0$. Synthesis tools will take care of these combinational details pretty well from an HDL code written either way. The mux-type description may be more readable. Mentioned for the sake of insights

Addition Examples (4-bit)

Arithmetic	Signed Interpretation	Unsigned Interpretation	N	Z	C	V
1111 + 1111	-1 + -1	15 + 15	1	0	1	0
1 1110	-2	14				
0001 + 0010	1 + 2	1 + 2	0	0	0	0
0 0011	3	3				
1010 + 1001	-6 + -7	10 + 9	0	0	1	1
1 0011	3	3				
0010 + 0111	2 + 7	2 + 7	1	0	0	1
0 1001	-7	9				
1111 + 0010	-1 + 2	15 + 2	0	0	1	0
1 0001	1	1				
0000 + 0000	0 + 0	0 + 0	0	1	0	0
0 0000	0	0				
0001 + 1111	1 + -1	1 + 15	0	1	1	0
1 0000	0	0				

Observe :

- Unsigned results are **wrong** when C=1
- Signed results are **wrong** when V=1

Overflow occurs when
 (+ve) + (+ve) = (-ve)
 or
 (-ve) + (-ve) = (+ve)

Obviously, N and Z
 cannot be 1
 simultaneously

Can Z and V be 1
 simultaneously?

Signed Addition

- For *unsigned* addition, carry=1 indicates that the result is wrong
- For *signed addition*,
 - Overflow occurs when (+ve) + (+ve) = (-ve) or (-ve) + (-ve) = (+ve)
 - If the sign bits (MSB) of the operands are the same, and is different from that of the result, there is an overflow
 - Overflow can never occur when you add two numbers of different signs (they tend to cancel each other)
 - Carry and overflow are unrelated
 - However, carry is sometimes called 'unsigned overflow'
- The hardware as well as the instruction used for performing both signed and unsigned addition are the same
 - The hardware doesn't know and doesn't care if you are dealing with signed or unsigned
 - Only the *interpretation of the result* and/or *the way the condition codes are used* are different

Note: Carry is still relevant for signed numbers in multi-word arithmetic when it is the result of addition/subtraction of lower significant words, but not when it is the result of addition/subtraction of the most significant word. Please see ADC/SBC instructions of ARM if you wish to understand more

Subtraction Examples (4-bit)

Arithmetic (Pen n Paper)	Arithmetic (Hardware)	Signed Interpretation	Unsigned Interpretation	N	Z	C	V
0 1111 - 0001 1110	1111 + 1110 + 1 1 1110	-1 - 1 -2	15 - 1 14	1	0	1	0
1 0001 - 1110 0011	0001 + 0001 + 1 0 0011	1 - -2 3	1 - 14 3	0	0	0	0
0 1010 - 0111 0011	1010 + 1000 + 1 1 0011	-6 - 7 3	10 - 7 3	0	0	1	1
1 0010 - 1001 1001	0010 + 0110 + 1 0 1001	2 - -7 -7	2 - 9 9	1	0	0	1
0 1111 - 1110 0001	1111 + 0001 + 1 1 0001	-1 - -2 1	15 - 14 1	0	0	1	0
0 0000 - 0000 0000	0000 + 1111 + 1 1 0000	0 - 0 0	0 - 0 0	0	1	1	0
0 0001 - 0001 0000	0001 + 1110 + 1 1 0000	1 - 1 0	1 - 1 0	0	1	1	0

↓
borrow

↓
carry

Carry = not(borrow)
This is a natural consequence of the hardware way of subtraction.

$A - B = A + (-B) =$
 $A + 2's(B) = A + B' + 1$
 [B' is one's complement of B, i.e., all bits of B flipped]

Observe :

- Unsigned results are **wrong** when $C=0$
- Signed results are **wrong** when $V=1$

Overflow occurs when
 $(+ve) - (-ve) = (-ve)$
 or
 $(-ve) - (+ve) = (+ve)$

Signed Subtraction

- For *unsigned* subtraction, borrow indicates that the result is wrong. However, note that ARM implements carry flag as $\text{not}(\text{borrow})$ for subtraction. Hence, if $\text{carry} = 0$, the result is wrong
 - Carry = borrow in Intel architecture, as Intel hardware flips the carry bit if the operation is subtraction. We follow ARM convention for now
- For *signed* subtraction,
 - Overflow occurs when $(+ve) - (-ve) = (-ve)$ or $(-ve) - (+ve) = (+ve)$
 - If the sign bits (MSB) of the operands are different, and the result has a sign same as that of the second operand, there is an overflow
 - Overflow can never occur in subtraction when operands are of the same sign (they tend to cancel each other)
 - Carry and overflow are unrelated
- The hardware as well as the instruction used for performing both signed and unsigned subtraction are the same – *only the interpretation of the result and/or the way the condition codes are used* are different
 - For example, in RV, for checking ' $<$ ', we should use *lt* with signed numbers, whereas it should be *ltu* for unsigned numbers

N, Z, C, V

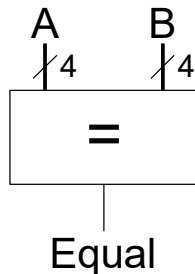
- $N = 1$ if *Result* is negative, i.e., $N = \text{Result}_{31}$
- $Z = 1$ if all of the bits of *Result* are 0
- $C = 1$ if
 - C_{out} of adder is 1 **AND**
 - Indication of unsigned addition result being wrong, unsigned subtraction result being right (but generated using the same logic for add and sub in the ARM convention of carry = !borrow)
- $V = 1$ if
 - A and *Result* have opposite signs ($A_{31} \oplus \text{Result}_{31}$) **AND**
 - If ADD : A and B have same sign ($\overline{A_{31} \oplus B_{31}}$)
 - If SUB : A and B have different signs ($A_{31} \oplus B_{31}$)
 - Indication of signed result being wrong for both add and sub (but generated with different logic for add and sub)
- For RV, NZCV is relevant only when ALU does subtraction; are not stored as flag bits for use by future instructions; but used within the same instruction for branch / slt variants
 - eq (Z), ne ($\bar{Z}$), lt ($N \oplus V$), ge ($\overline{N \oplus V}$), ltu ($\bar{C}$), geu (C) – why?

For ARM too, the condition code mnemonics such as EQ, NE, LT, GE, HI,... make sense (corresponds to flag combinations meaningfully) only following CMP/SUBS

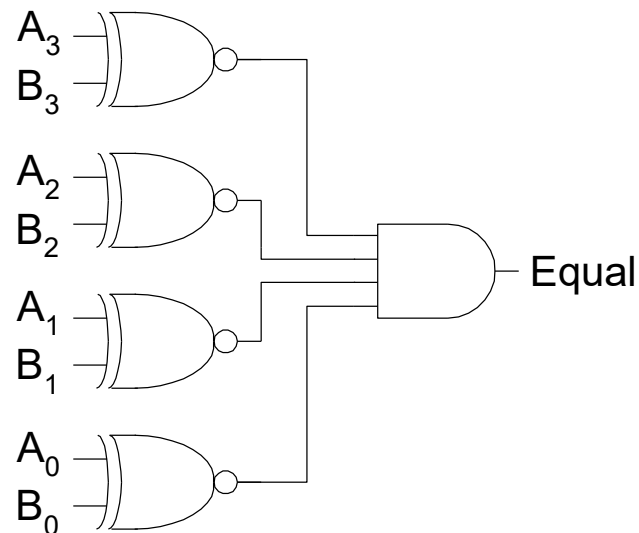
Comparator: Equality

- Comparison is generally is done using subtraction
- ALU need not contain dedicated comparison hardware
- However, equality check might still be required for other purposes
 - Pipeline hazard detection
 - Branch prediction

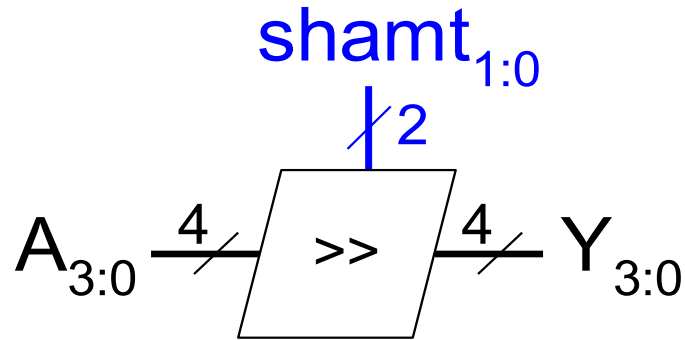
Symbol



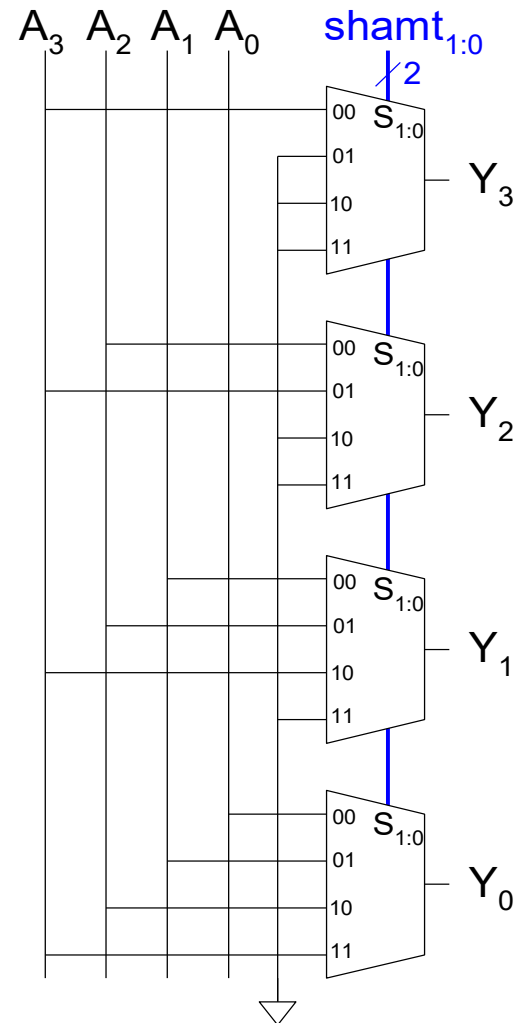
Implementation



Shifter (srl)

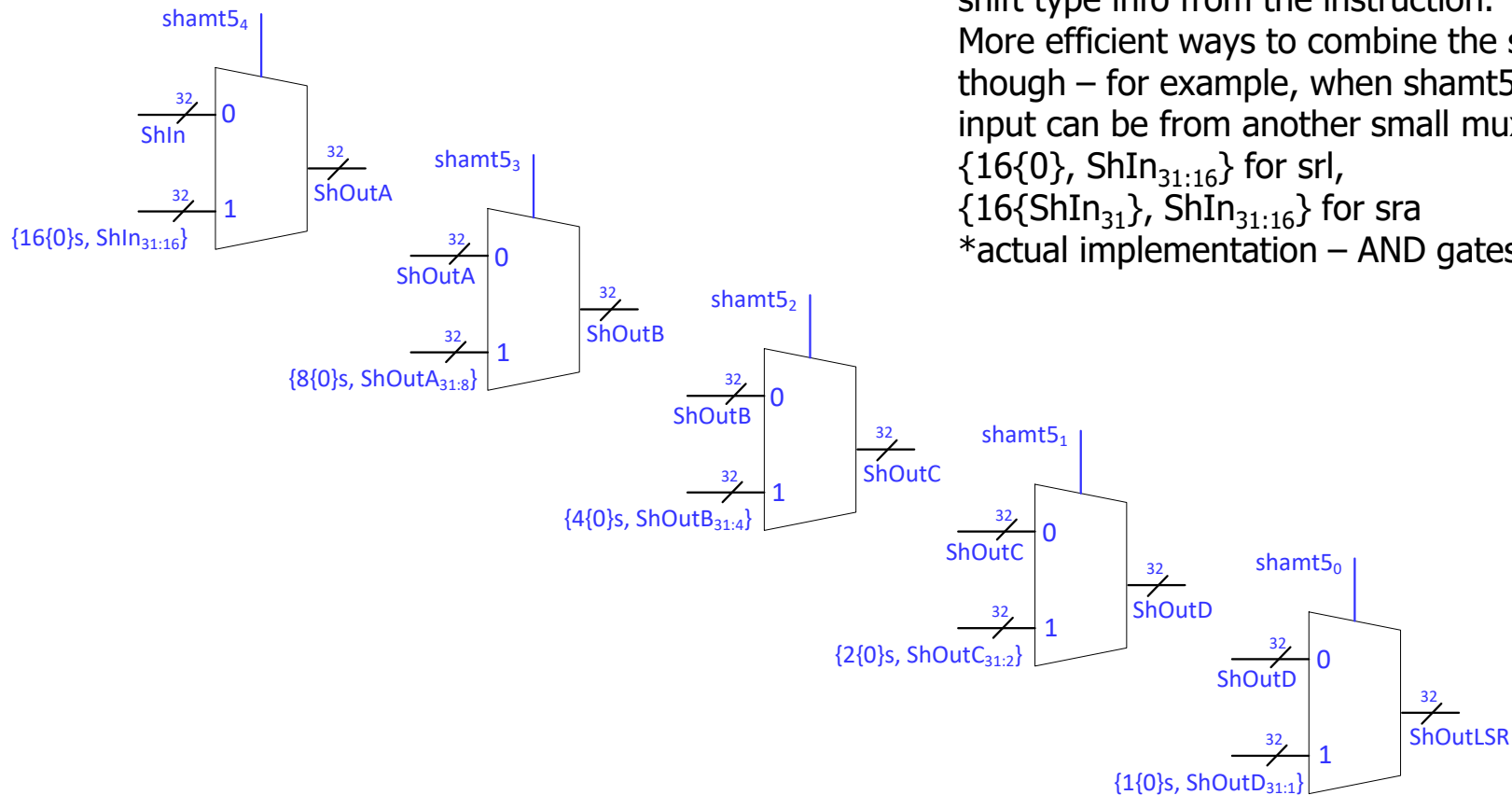


- High hardware usage for 32 bit shifting
 - 32 x 32-1 mux
- Such a shifter which can combinationally shift an input by an arbitrary amount is called a barrel shifter



Shifter

- Hardware-efficient shifter
 - 5 x 32 x 2-1 mux



sll, sra can also be implemented following a similar logic.

Rotates can also be done in a similar manner, though not supported as a base instruction in RV. The 3 different shifts (sll, sra, srl) can be combined using a multiplexer, controlled by the shift type info from the instruction.

More efficient ways to combine the shifts exist though – for example, when $shamt_4=1$, the input can be from another small mux* that gives $\{16\{0\}, ShIn_{31:16}\}$ for srl, $\{16\{ShIn_{31}\}, ShIn_{31:16}\}$ for sra

*actual implementation – AND gates

Multipliers

- **Partial products** formed by multiplying a single digit of the multiplier with multiplicand
- **Shifted partial products** summed to form result

Decimal

$$\begin{array}{r} 230 \\ \times 42 \\ \hline 460 \\ + 920 \\ \hline 9660 \end{array}$$

$$230 \times 42 = 9660$$

Binary

multiplicand
multiplier
partial
products
result

$$\begin{array}{r} 0101 \\ \times 0111 \\ \hline 0101 \\ 0101 \\ 0101 \\ + 0000 \\ \hline 0100011 \end{array}$$

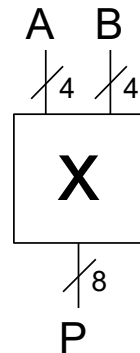
$$5 \times 7 = 35$$

Multiplier (4 x 4)

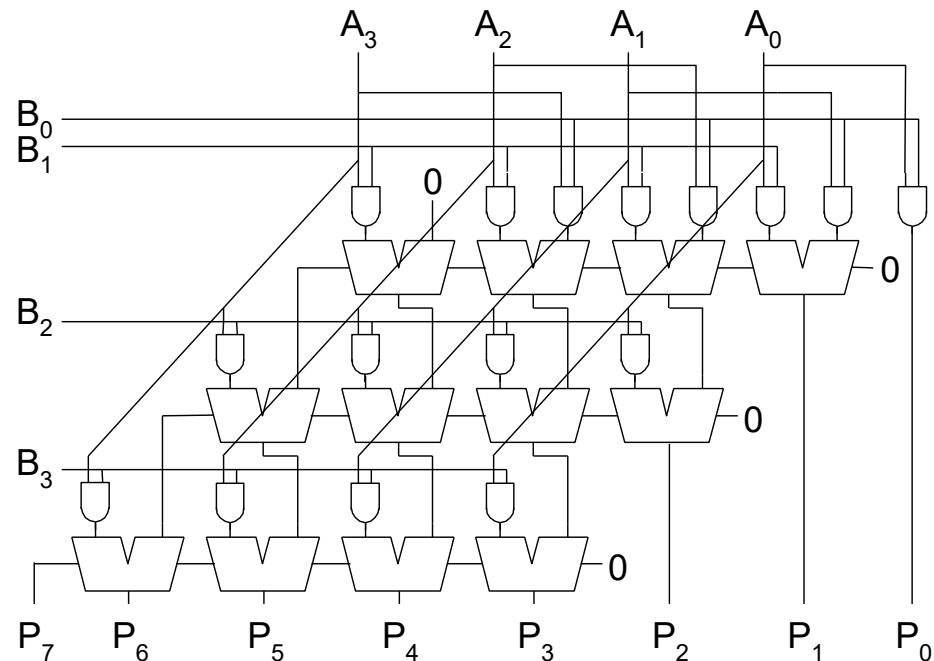
FPGAs usually have some for a single cycle multiplier built in. Read up about DSP48E of AMD FPGAs

■ Array multiplier

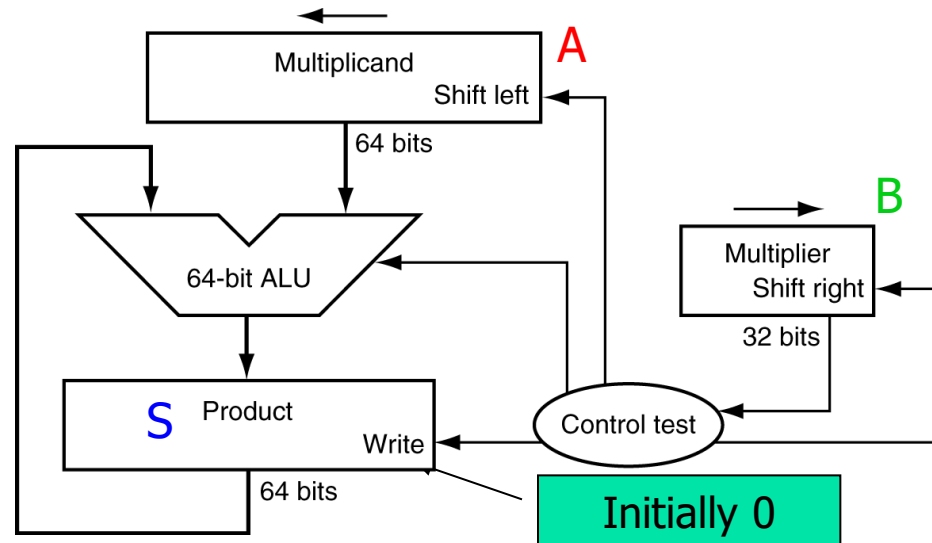
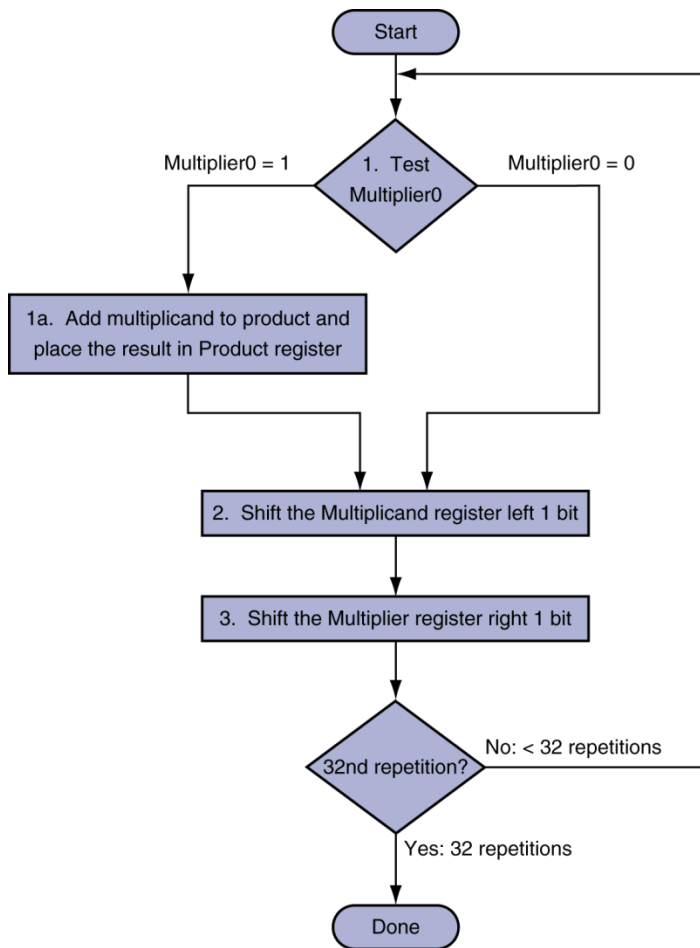
- Pro : combinational (single cycle)
- Cons: Very high hardware usage, delay (likely in the critical path)



$$\begin{array}{r}
 \begin{array}{cccc}
 & A_3 & A_2 & A_1 & A_0 \\
 \times & B_3 & B_2 & B_1 & B_0 \\
 \hline
 & A_3B_0 & A_2B_0 & A_1B_0 & A_0B_0 \\
 & A_3B_1 & A_2B_1 & A_1B_1 & A_0B_1 \\
 & A_3B_2 & A_2B_2 & A_1B_2 & A_0B_2 \\
 + & A_3B_3 & A_2B_3 & A_1B_3 & A_0B_3 \\
 \hline
 P_7 & P_6 & P_5 & P_4 & P_3 & P_2 & P_1 & P_0
 \end{array}
 \end{array}$$



Sequential Multiplier



Let $S=000000$, $A=011$, $B=101$ (initial values)

#1: B_0 (orig B_0)=1

add: $S = S + A = 000000 + 000011 = 000011$

$A=A<<1$; Thus $A = 000110$

$B=B>>1$; Thus $B = 010$

#2: B_0 (orig B_1)=0; no addition required;

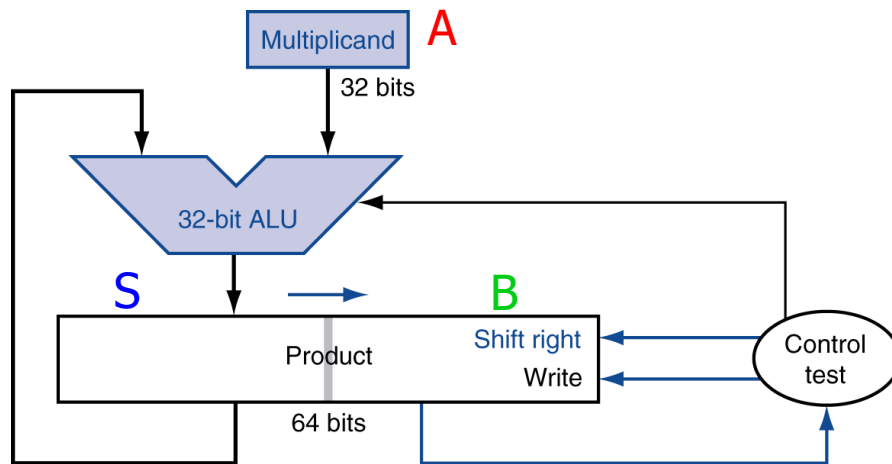
$A=A<<1$; Thus $A = 001100$

$B=B>>1$; Thus $B = 001$

#3: B_0 (orig B_2)=1

add: $S = S + A = 000011 + 001100 = 001111$

Improved Sequential Multiplier



Idea is to always align **A** to the “high side” (3 most significant bits of **S**) and then perform addition; Shift **S** to the right and do not shift **A**

Least significant bits of **S** populated with **B**, which keeps getting shifted to the right with **S** (avoids the need for an extra register for **B**)

Let **S**=000**101** (000**B**), **A**=0**11** (initial values)

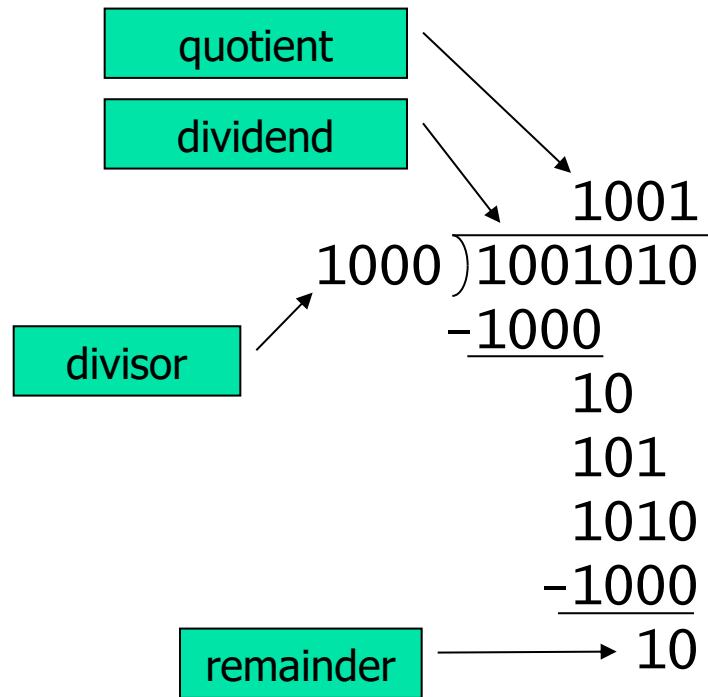
#1: $S_0(\text{orig } B_0)=1$; add: **S** = **S** + **A**000 = 000**101** + 0**11**000 = 0**11****101**
S=**S**>>1; Thus **S**=00**11****10**

#2: $S_0(\text{orig } B_1)=0$; no addition required;
S=**S**>>1; Thus **S**=000**111**

#3: $S_0(\text{orig } B_2)=1$; add: **S** = **S** + **A**000 = 000**111**+ 0**11**000 = 0**11****111**;
S=**S**>>1; Thus **S**=00**11****11**, which is the final result

The above block diagram and example are just to illustrate the idea. Slight adjustments are needed to get it correct for cases where there is a carry from 32-bit addition

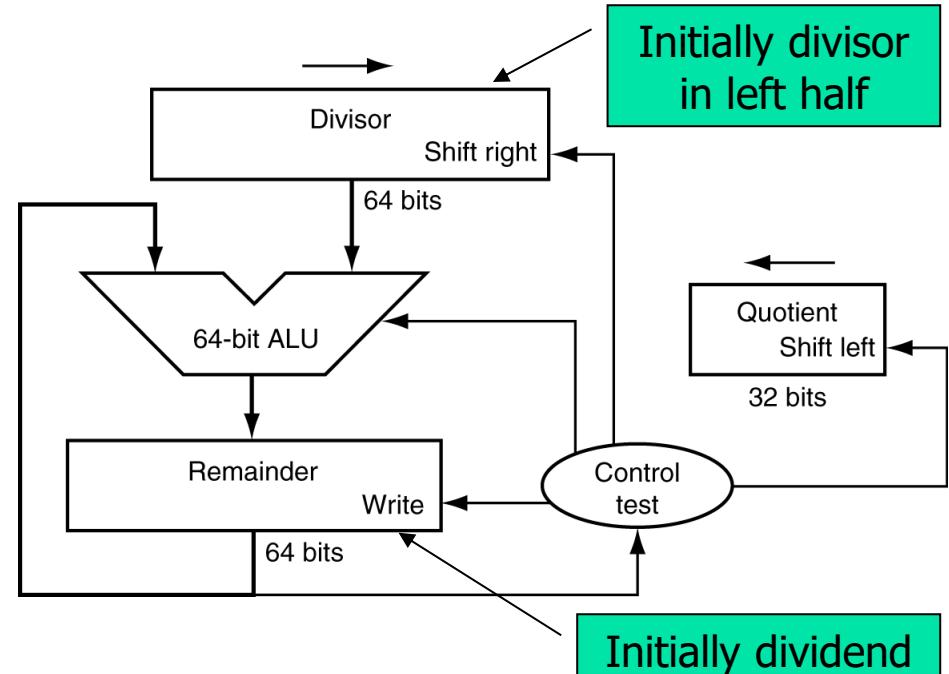
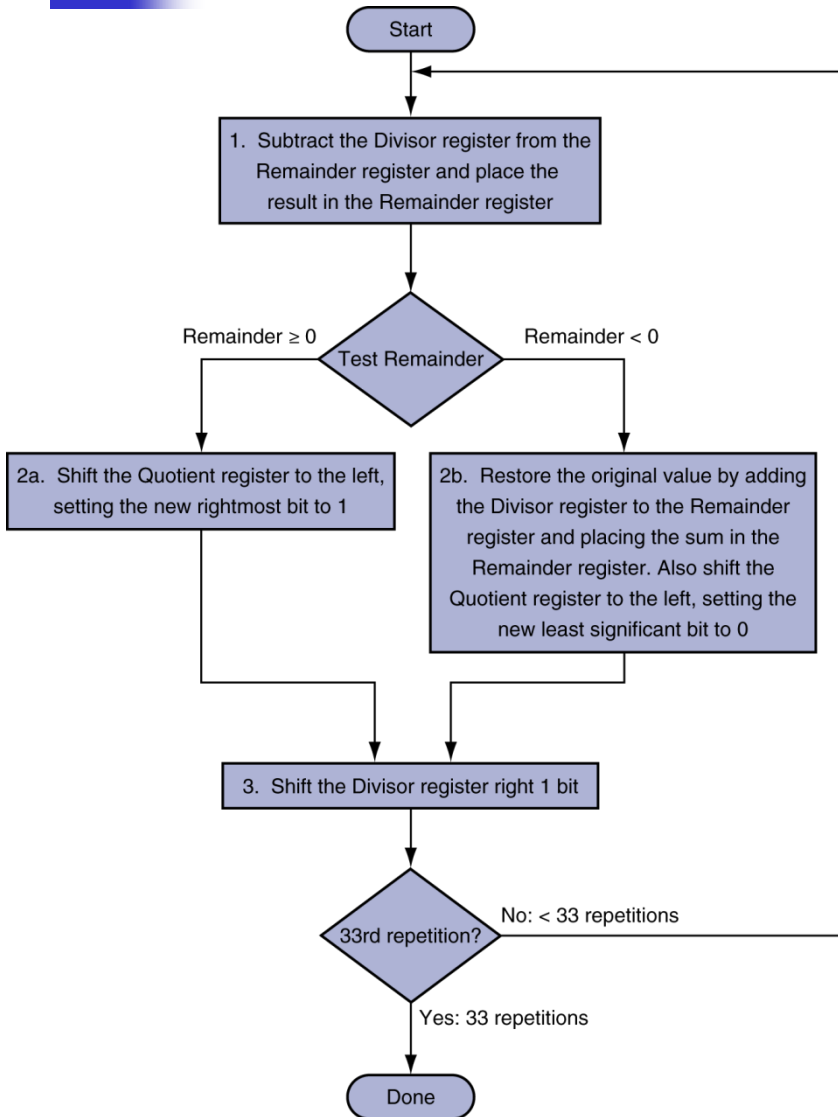
Division



n -bit operands yield n -bit quotient and remainder

- Check for 0 divisor
- Long division approach
 - If divisor $\leq$ dividend bits
 - 1 bit in quotient, subtract
 - Otherwise
 - 0 bit in quotient, bring down next dividend bit
- Restoring division
 - Do the subtract, and if remainder goes < 0 , add divisor back

Division Hardware



- More efficient hardware (similar to that for multiplication) possible
- Better schemes such as non-restoring division exists
 - Left as self-study
- Multiplier and Divider can share most of the hardware

Example

- Steps involved when 0000 0111 is divided by 0010

Iteration	Step	Quotient	Divisor	Remainder
0	Initial values	0000	0010 0000	0000 0111
1	1: Rem = Rem – Div	0000	0010 0000	①110 0111
	2b: Rem < 0 $\Rightarrow$ +Div, sll Q, Q0 = 0	0000	0010 0000	0000 0111
	3: Shift Div right	0000	0001 0000	0000 0111
2	1: Rem = Rem – Div	0000	0001 0000	①111 0111
	2b: Rem < 0 $\Rightarrow$ +Div, sll Q, Q0 = 0	0000	0001 0000	0000 0111
	3: Shift Div right	0000	0000 1000	0000 0111
3	1: Rem = Rem – Div	0000	0000 1000	①111 1111
	2b: Rem < 0 $\Rightarrow$ +Div, sll Q, Q0 = 0	0000	0000 1000	0000 0111
	3: Shift Div right	0000	0000 0100	0000 0111
4	1: Rem = Rem – Div	0000	0000 0100	①000 0011
	2a: Rem $\geq$ 0 $\Rightarrow$ sll Q, Q0 = 1	0001	0000 0100	0000 0011
	3: Shift Div right	0001	0000 0010	0000 0011
5	1: Rem = Rem – Div	0001	0000 0010	①000 0001
	2a: Rem $\geq$ 0 $\Rightarrow$ sll Q, Q0 = 1	0011	0000 0010	0000 0001
	3: Shift Div right	0011	0000 0001	0000 0001

Signed Multiplication / Division

11x	Unsigned interpretation
10	
0000	3 x 2 = 6
0110	Signed interpretation
0110	-1 x -2 = +6

1111x	Corrected signed multiplication
1110	
0000	Need to do sign-extension!
1110	
1100	
1000	
0010	

- When multiplying two n-bit numbers, the least significant n-bits are the same irrespective of signed / unsigned multiplication. MUL (ARM/RV) instruction gives this 32-bit result
 - The most as well as least significant 32 bits can be obtained via a single SMULL/UMULL in ARM
 - In RV, in addition to mul, we need separate mulh/mulu instructions. This need not necessitate separate multiplications if the hardware is capable of *macro-op fusion*
- For division, signed/unsigned always matter
- There are more efficient signed multiplication techniques – read up!
- One simple trick – convert negatives to positives before multiplication / division, and negate the result if the original operands are of different signs

Numbers with Fractions

- Two common notations
 - Fixed-point: binary point fixed
 - Floating-point: binary point floats to the right of the most significant 1
- Fixed point
 - Binary point is implied - the number of integer and fraction bits must be agreed upon beforehand by the parties using it
 - Notations not so standard
 - The position of the binary point affects the *range* (difference between the largest and smallest representable numbers) and *precision* (smallest possible difference between any two numbers)
 - Example : 6.75 using 4 integer bits and 4 fraction bits

01101100

0110.1100

$$2^2 + 2^1 + 2^{-1} + 2^{-2} = 6.75$$

Signed Fixed Point

- Representation using Sign&Mag or 2's compl.
- Example: Represent -7.510 using 4 integer and 4 fraction bits
 - Sign/magnitude
 - 11111000
 - 2's complement
 1. +7.5: 01111000
 2. Invert bits: 10000111
 3. Add 1 to LSB: +1

10001000
- Addition is exactly the same as integer addition, position of point is unaffected
- After multiplication / division using standard integer operations, the position of point should be adjusted by shifting the point to the left (i.e., shifting the result to the right) / right (i.e., shifting the result to the left) by the number of fraction bits (why?)



Fixed Point

It is also possible to extend the fixed-point concept to have a range wider than 0 to $2^{n_bits}-1$ (unsigned; similar idea for signed too). The position of the point just needs to be further to the right of the LSB. This will reduce the precision to be below integer precision though

- (+) It's simple. The same hardware that does integer arithmetic can do fixed point arithmetic
- Supported by some DSPs explicitly. HLS tools also have arbitrary precision fixed point classes
- (+) The programmer can use integers with an implicit fixed point in high-level software programming languages
 - ints are just fixed-point numbers with the binary point to the immediate right of LSB (b0)
- (-) No easy way to decide where the point should be
 - Sometimes you need range, sometimes you need precision
 - The more you have of one, the less of the other for a given number of bits
- (-) There is no way of knowing the existence of the binary point except through agreement of those interpreting the number

Fixed Point Multiplication Example

- Fixed point has an implicit scale factor, which is $2^{\text{number of fraction bits}}$
- Say, we are following an 8-bit format with 6 integer and 2 fraction bits. This means a CPU with word-size of 8
- $a = (1.5)_{10} = (000001.10)_2$ in our mind/paper. It is stored in registers/memory as $A = 00000110$
 - The original number is scaled by 2^2 to be represented/stored, resulting in the shifting of the point to the right by 2 positions
 - The position of the point is implicit, i.e., the point is removed
- $b = (2.5)_{10} = (000010.10)_2$. $B = 00001010$
- $c = (1.5 * 2.5 = 3.75)_{10} = (000001.10 * 000010.10 = 000011.11)_2$. To maintain the same format, we want $C = 00001111$
- The integer ALU will do $A * B = 00000110 * 00001010 = 00111100$
 - Note: The scale factor gets squared ($2^2 * 2^2$)
- To get the expected answer (C), we do $00111100 \gg 2 = 00001111$
 - This is essentially division by the scale factor, to remove the extra $*2^2$

Fixed Point Multiplication Example ...

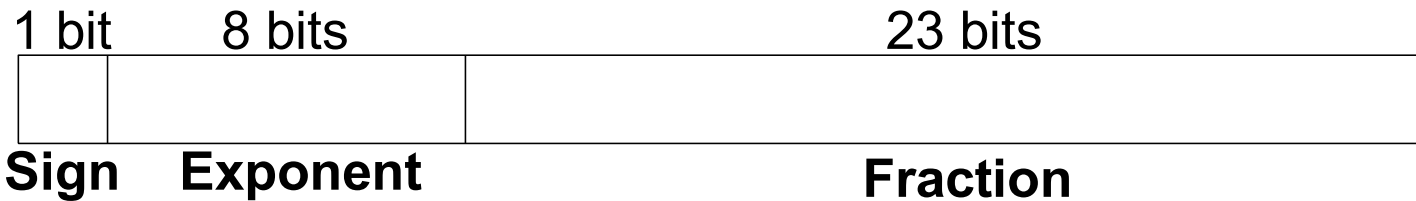
- In the example above, we assumed a short multiply, i.e., $n\text{-bit} \times n\text{-bit} = n\text{-bit}$, which is fine for this case as the numbers are much smaller than the range possible with n bits
- If this is not the case, we may need to do a long multiply ($2n\text{-bit}$ results). The result will be over 2 registers
- We will need to shift the entire $2n\text{-bit}$ result to the right, and trim off an appropriate number of MSBs to fit into n bits to maintain the format
- If trimming MSBs causes an incorrectness of the results, the result is too big (beyond the range) for the representation system (doesn't fit into n bits), i.e., there is an *overflow* (in the generic sense of the term)
 - This happens when, $a = (10.5)_{10} = (001010.10)_2$ and $b = (20.5)_{10} = (010100.10)_2$
 - The expected result, $c = (215.25)_{10}$ is too big for 6 integer and 2 fraction bits
- If $a = (1.5)_{10}$ and $b = (2.25)_{10}$, $c = (3.375)_{10}$, we have too few fraction bits to represent the result accurately, i.e., beyond our precision
- This is a case of *underflow* and requires rounding (will see later)

Floating-Point Numbers

- Binary point 'floats' unlike fixed point – the position is explicitly encoded (as Exponent) in the representation format
- Scientific notation usually used
- The IEEE 754 floating-point standard
 - a number is written as

Floating-Point representation has both logarithmic and linear parts, unlike integer/fixed-point which is purely linear. Allows very large ($\sim \pm 2^{127}$) and very small ($\sim \pm 2^{-127}$) numbers to be represented with the same format.

$$\pm 1.\text{Fraction} \times 2^{(\text{Exponent}-127)}$$



- The '1' to the left of the point is implicit/hidden
- 127 is the 'bias' so that the Exponent field is always positive (known as excess-127 format)
- Significand is also known as Mantissa and some other terms; may or may not include the hidden bit depending on context

Floating Point Example

Write -58.25_{10} in floating point (IEEE 754)

1. Convert decimal to binary:

$$58.25_{10} = 111010.01_2$$

2. Write in binary scientific notation:

$$1.1101001 \times 2^5$$

3. Fill in fields:

Sign bit: 1 (negative)

8 exponent bits: $(127 + 5) = 132 = 10000100_2$

23 fraction bits: 110 1001 0000 0000 0000 0000

1 bit	8 bits	23 bits
1	10000100	110 1001 0000 0000 0000 0000
Sign	Exponent	Fraction

in hexadecimal: 0xC2690000

IEEE 754 : Special Cases

Single precision already provides a very good range of approx. $\pm 2^{127}$. Double precision provides a range of approx. $\pm 2^{1023}$ with an increase in exponent bits from 8 to 11. Doubling exponent bits from 8 to 16 would have been unnecessary for most applications

Sign	Exponent	Significand	'Expected' Interpretation	Actual Meaning
X	00000000	all zero	$\pm 1.000000000000000000000000 \times 2^{-127}$	0
0	11111111	all zero	$+1.000000000000000000000000 \times 2^{128}$	∞
1	11111111	all zero	$-1.000000000000000000000000 \times 2^{128}$	$-\infty$
X	11111111	non-zero	$\pm 1. \text{xxxxxxxxxxxxxxxxxxxxxxxxxx} \times 2^{128}$ where xs are not all 0s	NaN

- Double-Precision (64-bit)
 - 1 sign bit, 11 exponent bits, 52 fraction bits. Bias = 1023
- Half-Precision (16-bit)
 - 1 sign bit, 5 exponent bits, 10 fraction bits. Bias = 15
 - Popular in machine learning / neural networks
 - Supported by modern GPUs (e.g., Nvidia Pascal and newer) and CPU ISAs/ISA extensions such as ARMv8.1-M, x86 AVX-512, **RISC-V Zfh**, etc.
- Smaller and flexible 'mini-float' representations, as low as 8 bits, exist

In applications such as machine learning – doing more computations with less precision is often favoured. Google TPU is 8-bit!



Overflow and Underflow

- Overflow: number too large to be represented (insufficient range)
 - Choice between saturation (computationally costlier, but inaccuracy is more graceful) and wrapping
- Underflow: number too small to be represented (insufficient precision)
 - Basic idea : compute exact result and make it fit into desired precision
 - Rounding modes: Down, Up, Toward zero, To nearest
- e.g.,: round 1.100101 (1.578125) to only 3 fraction bits
 - Down: 1.100
 - Up: 1.101
 - Toward zero: 1.100
 - To nearest: 1.101 (1.625 is closer to 1.578125 than 1.5 is)
 - Lower error accumulation but computationally costlier



Floating Point Addition : Steps

1. Extract exponent and fraction bits
2. Prepend leading 1 to form the significand
3. Compare exponents
4. Shift the smaller significand to the right
5. Add significands
6. Normalize significand and adjust exponent if necessary
7. Round result
8. Assemble exponent and fraction back into floating-point format

Floating Point Addition : Example

- Add the following floating-point numbers
0x3FC00000, 0x40500000

1. Extract exponent and fraction bits

1 bit	8 bits	23 bits
0	01111111	100 0000 0000 0000 0000 0000
Sign	Exponent	Fraction

1 bit	8 bits	23 bits
0	10000000	101 0000 0000 0000 0000 0000
Sign	Exponent	Fraction

For first number (N1): $S = 0, E = 127, F = .1$

For second number (N2): $S = 0, E = 128, F = .101$

2. Prepend leading 1 to form significand

N1: 1.1

N2: 1.101

Floating Point Addition : Example

3. Compare exponents

$127 - 128 = -1$, so shift N1 right by 1 bit

4. Shift smaller significand if necessary

shift N1's significand: $1.1 \gg 1 = 0.11 \ (\times 2^1)$

5. Add significands

$$\begin{array}{r} 0.11 \times 2^1 \\ + \quad 1.101 \times 2^1 \\ \hline 10.011 \times 2^1 \end{array}$$

Floating Point Addition : Example

6. Normalize significand and adjust exponent if necessary

$$10.011 \times 2^1 = 1.0011 \times 2^2$$

7. Round result

No need (fits in 23 bits)

8. Assemble exponent and fraction back into floating-point format

$$S = 0, E = 2 + 127 = 129 = 10000001_2, F = 001100..$$

1 bit	8 bits	23 bits
0	10000001	001 1000 0000 0000 0000 0000
Sign	Exponent	Fraction

in hexadecimal: 0x40980000

Floating Point Multiplication

- Unpack the numbers. Assume $A=0_0111_110$ and $B=0_1001_010$
- Sign of result = xor of signs of the numbers = $0 \text{ xor } 0 = 0$
- Add the two exponents using integer math. Subtract the bias from the result
 - $E=0111+1001-0111=1001$
- Multiply the significands (incl. hidden bits) via integer math
 - $S=1.110*1.010=10.001100$
- Normalize the significand and adjust exponent if necessary
 - $S=1.00011, E = 1010$
- Round the significand
 - $S= 1.000$ (assuming rounding mode is truncation)
- Pack it back. Result = 0_1010_000



Floating Point - Considerations

- No need to stick to the prescribed number of bits for intermediate results – need to convert it back to the form only when writing result to a memory/register accessible by other hardware/software
- FPGA multipliers are larger sized than 'usual' – e.g., 25x18 instead of 16x16 to allow for higher intermediate precision
- Many floating point units have internal registers that are wider than 'usual'
- Floating point has inexact representation even for some simple fractions (e.g., 0.1)
 - Can lead to non-intuitive behaviour. Good idea not to check for equality ($a==b?$), - different rounding modes could give different representations for the same value
- Hard to make fast in hardware
 - Numerical analysts predominated over hardware designers in defining standard
- Floating point division is an especially complex operation
 - Might be better to do $x*(1/y)$ instead of x/y if the same y is used for many back-to-back divisions



Summary

- We saw circuits for
 - Ripple carry and carry look-ahead adders
 - Simple ALU and condition codes
 - Barrel shifters
 - Multipliers – combinational and sequential
 - Division

- Numbers with fractions
 - Fixed point and floating point
 - Fixed point representation and arithmetic
 - Floating point format representation and arithmetic

Comparisons

- $A < B \Rightarrow B > A$
 - bgt t1, t2, label is encoded as blt t2, t1, label
- Similarly
 - ble t1, t2, label is encoded as bge t2, t1, label
 - bgtu t1, t2, label is encoded as bltu t2, t1, label
 - bleu t1, t2, label is encoded as bgeu t2, t1, label
- $(A < B) == !(A \geq B)$
 - It is $(N \oplus V)$; ge is $\overline{(N \oplus V)}$
 - Itu is $(\bar{C})$, geu is (C)
- A hypothetical sle t2, t1, t0 (there is no sle instruction or pseudoinstruction) can be implemented as (we need 2 instrns)
 - slt t2, t0, t1 // t2 = (t0 < t1) == !(t0 >= t1) == !(t1 <= t0)
 - xori t2, t2, 1 // xor with 1 is negation; t2 = (t1 <= t0)

lt vs ltu

- $ltu(\bar{C})$: When unsigned $A < B$, $A - B$ causes a borrow, i.e., no carry
- $lt(N \oplus V)$: When signed $A < B$
 - $A - B$ gives a negative result (N) correctly if no overflow ($\bar{V}$). i.e., $N \cdot \bar{V}$
 - $A - B$ causes a spurious positive result ($\bar{N}$) if A is negative, B is positive and there is an overflow (V): $(-ve) - (+ve) = (+ve)$. i.e., $\bar{N} \cdot V$
 - Hence, lt is $N \cdot \bar{V} + \bar{N} \cdot V = N \oplus V$
- When signed $A \geq B$
 - $A - B$ causes a spurious negative result (N) if A is positive, B is negative and there is an overflow (V): $(+ve) - (-ve) = (-ve)$. i.e., $N \cdot V$
 - $A - B$ gives a zero/positive result ($\bar{N}$) correctly if no overflow ($\bar{V}$). i.e., $\bar{N} \cdot \bar{V}$
 - Hence, ge is $N \cdot V + \bar{N} \cdot \bar{V} = \overline{(N \oplus V)}$

slt, slti, sltu, sltiu

- What are the values of x4-x8 after executing the programme below?

```
li x1, 1
```

```
li x2, -1
```

```
li x3, 0xFFFFFFFF
```

```
sub x4, x3, x2    //x4 = 0 as x3 = x2 = 0xFFFFFFFF
```

```
slt x5, x3, x1    //x5 = 1 as x3 (-1) < x1 (1)
```

```
sltu x6, x3, x1   //x6 = 0 as x3  $(2^{32}-1)^*$   $\nless$  x1 (1)
```

```
slti x7, x3, -2   //x7 = 0 as x3 (-1)  $\nless$  (-2)
```

```
sltiu x8, x3, -2  //x8 = 0 as x3  $(2^{32}-1)$   $\nless$   $(2^{32}-2)^{\$}$ 
```

*the content of x3 is 0xFFFFFFFF, which is -1 if interpreted as signed, $2^{32}-1$ if interpreted as unsigned. sltu/sltiu interprets operands as unsigned

$\$$ the assembler encodes the immediate -2 as 0xFFE, and as RISC-V always MSB-extends the immediate (i.e., duplicates MSB, i.e., bit 11 of the immediate is duplicated into bits 31:12), the operand is 0xFFFFFFFEE. However, since sltiu interprets the operand as unsigned, 0xFFFFFFFEE will be interpreted as $2^{32}-2$

Multi-word Arithmetic

- Say, we must do $N3 = N1 + N2$ where all are 64-bit. $N1$ is spread over $a1$ (MSW) and $a0$ (LSW). Similarly, $N2$ is in $a3$ and $a2$. $N3$ should be in $a1$ and $a0$. Write RV and ARM codes
- RV
 - add $a0, a0, a2$ // $N3_LSW = N1_LSW + N2_LSW$
 - add $a1, a1, a3$ // $N3_MSW = N1_MSW + N2_MSW$
 - sltu $t0, a0, a2$ // if $N3_LSW < N1_LSW$ (or $N2_LSW$), a carry has occurred. Why sltu works irrespective of signed or unsigned addition? Why is comparison with either $N1_LSW$ or $N2_LSW$ good enough?
 - add $a1, a1, t0$ // $N3_MSW = N3_MSW + \text{carry}$
- ARM
 - ADDS $R0, R2$
 - ADC $R1, R3$
- Does that mean RV is slower than ARM?
 - Not necessarily. Depends on microarchitecture
 - A superscalar processor will likely execute first 2 RV instrns in parallel
 - The third and fourth instrns could possibly be fused (*macro-op fusion*)

Note: add $a0, a2$ is not ok in RV. Need to specify destination explicitly

Multi-word Arithmetic ...

- Why does `sltu` work irrespective of signed or unsigned addition?
 - Whether the numbers are signed or unsigned makes a difference only in how we use $N1_{63}$, $N2_{63}$, $N3_{63}$, and the final carry C_{64}
 - The carry between the bit positions other than the final carry is still relevant for addition irrespective of whether we are dealing with signed or unsigned numbers. Just that this is handled by the hardware (cascading) for all bits other than between LSW and MSW
 - To find the carry between LSW and MSW, we have to do an *unsigned* comparison (remember, only unsigned comparisons check carry)
- Why is comparison with either $N1_LSW$ or $N2_LSW$ good enough?
 - If there is no carry : $N3_LSW = N1_LSW + N2_LSW$
This implies $N3_LSW \geq N1_LSW$ **and** $N3_LSW \geq N2_LSW$
(LSWs considered unsigned irrespective of the signedness of $N1$ and $N2$)
 - If there is carry : $2^{32} + N3_LSW = N1_LSW + N2_LSW$
(carry as a place value of 2^{32} , i.e., 2^{32} needs to be passed from LSW to MSW). Hence, $N3_LSW < N1_LSW$. Else, $N2_LSW \geq 2^{32}$, which is not possible with 32 bits; **and** $N3_LSW < N2_LSW$. Else, $N1_LSW \geq 2^{32}$, which is not possible with 32 bits
 - The following cases are not possible:
 $N3_LSW < N2_LSW$ and $N3_LSW \nless N1_LSW$
 $N3_LSW < N1_LSW$ and $N3_LSW \nless N2_LSW$
 - This means carry can be checked by comparing $N3_LSW$ with $N1_LSW$ or $N2_LSW$

Sign-extension (4-bit to 8-bit)

Input	Operation	Output	Remarks
0b1110 -2 (%d) 14 (%u)	Zero extension to 8 bits: fill 4 MSBs with 0s	00001110 14 (%d) 14 (%u)	Wrong for signed
0b0110 6 (%d) 6 (%u)	Zero extension to 8 bits: fill 4 MSBs with 0s	00000110 6 (%d) 6 (%u)	Correct for signed only as it is positive
0b0110 6 (%d) 6 (%u)	MSB extension to 8 bits: duplicate MSB[3] to [7:4]	00000110 6 (%d) 6 (%u)	Correct for unsigned only as MSB is 0
0b1110 -2 (%d) 14 (%u)	MSB extension to 8 bits: duplicate MSB[3] to [7:4]	11111110 -2 (%d) 254 (%u)	Wrong for unsigned